



Population Based Augmentation : Efficient Data Augmentation Policy Schedules

최석림



Computational Data Science LAB



CONTENTS

1. Intro
 2. Approach
 3. Experiment
 4. Conclusion
- 
- 

01 | Intro

- Data Augmentation은 cropping, translation, flipping등이 많이 활용됨
- Data Augmentation은 모델의 over-fitting을 예방(일반화)
- 그러나, 대부분의 모델에서는 일반적인 방식의 증강기법만 사용하고 있음



Original



Horizontal Flip



Pad & Crop

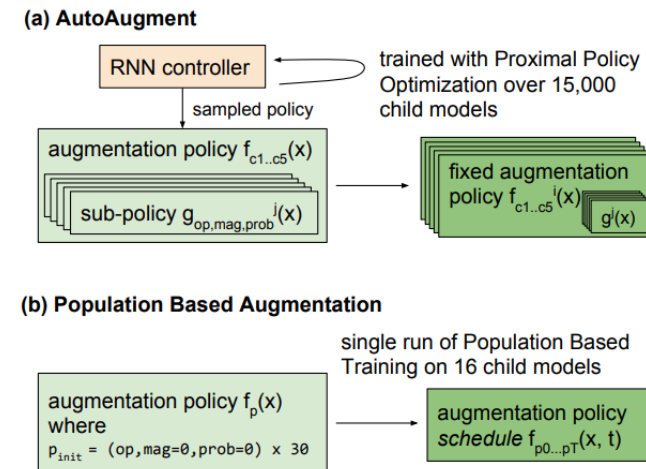


Rotate

01 | Intro

- AutoAugment(AA)은 증강기법의 자동탐색을 통해 SOTA를 달성하였으나, 막대한 탐색비용을 요구하며 학습기간 동안 고정된 증강 정책만 학습하도록 설계되어 있음
- 저자가 제안하는 Population Based Augmentation(PBA)는 학습기간 동안 증강 정책이 변화하는 스케줄을 학습하며, 더 적은 학습량으로 비슷하거나 우월한 성능을 발휘

Dataset	Value	Previous Best	AA	PBA
CIFAR-10	GPU Hours	-	5000	5
	Test Error	2.1	1.48	1.46
CIFAR-100	GPU Hours	-	0*	0*
	Test Error	12.2	10.7	10.9
SVHN	GPU Hours	-	1000	1
	Test Error	1.3	1.0	1.1



02 | Approach

Why Augmentation Schedule?

- 증강 정책을 고정할 경우 증강 정책의 효과를 확인하기 위해서는 자녀모델의 학습이 완료되어야함(대량의 계산 비용 초래)
반면 제안한 모델은 이전의 학습결과를 재사용하기 때문에 훨씬 경제적임
- 훈련 시점마다 모델에 필요한 증강 정책이 다를 수 있음
 - ✓ 특정 시점에 필요한 증강기법은 또 다른 시점에서는 필요 없을 수 있기 때문에 증강정책을 고정할 경우 유용한 증강기법이 무시되는 문제가 발생할 수 있음
 - ✓ 스케줄 탐색 공간은 매우 방대하지만, 효과적인 스케줄은 Smooth하기 때문에 PBT와 같은 기법에서는 좋은 정책을 쉽게 찾아낼 수도 있음

02 | Approach

Policy Search Space

- 15개의 증강방식 활용
- 각 연산은 두번씩 존재 가능(AA기법과 동일)
 - ☞ 30개의 tuple(15 x 2)
- 각 tuple에서 학습하는 hyperparameter는 확률과 강도 두가지
 - ✓ 확률 : 10% 단위 11가지
 - ✓ 강도 : 0 ~ 9까지 10가지
 - ☞ 설정 조합 : $(11 \times 10)^{30} = 1.75 \times 10^{61}$
 - * AA : 2.8×10^{32}

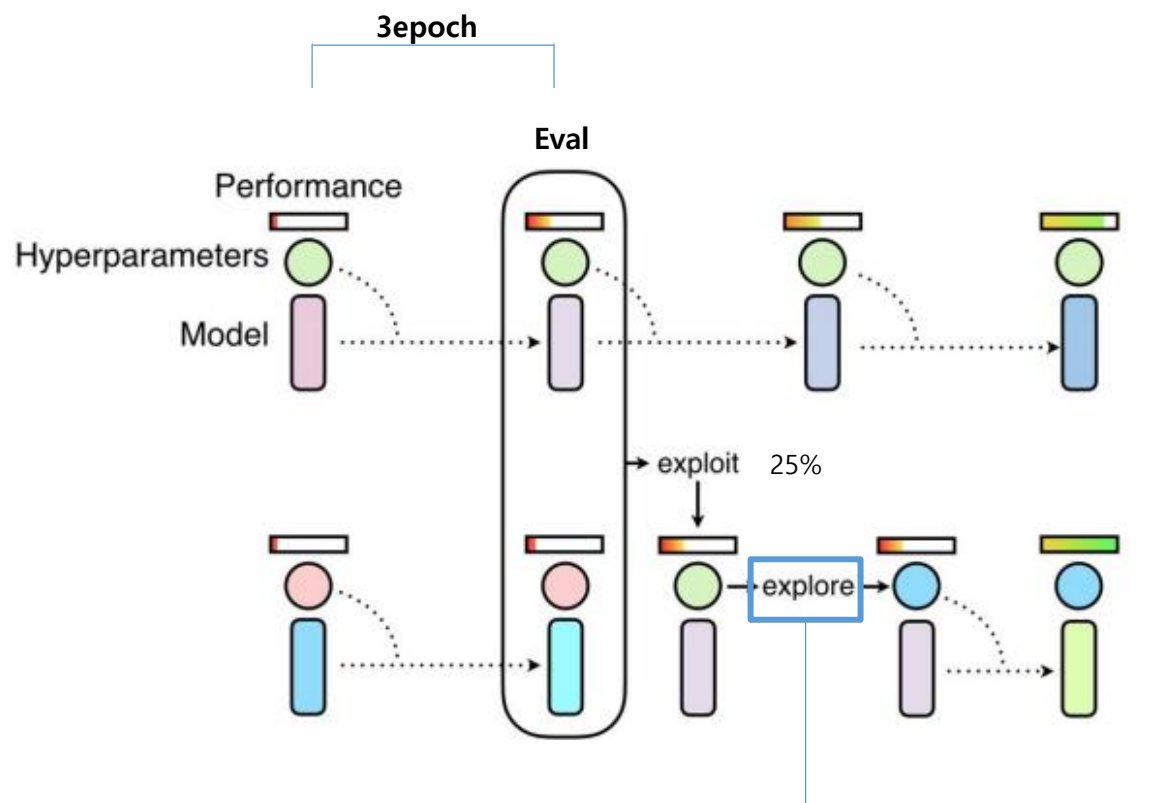
Algorithm 1 The PBA augmentation policy template, the parameters of which are optimized by PBT. The parameter vector is a vector of $(op, prob, mag)$ tuples. There are two instances of each op in the vector, and this parameter cannot be changed. PBT learns a schedule for the $prob$ and mag parameters during the course of training a population of child models.

Input: data x , parameters p , [list of $(op, prob, mag)$]
Shuffle parameters
Set $count = [0, 1, 2]$ with probability $[0.2, 0.3, 0.5]$
for $(op, prob, mag)$ in p **do**
 if $count = 0$ **then**
 break
 end if
 if $\text{random}(0, 1) < prob$ **then**
 $count = count - 1$
 $x = op(x, mag)$
 end if
end for
Return x

02 | Approach

PBA Implementation

- PBT 실험 형식에 맞추어 구현



Algorithm 2 The PBA explore function. Probability parameters have possible values from 0% to 100% in increments of 10%, and magnitude parameters have values from 0 to 9 inclusive.

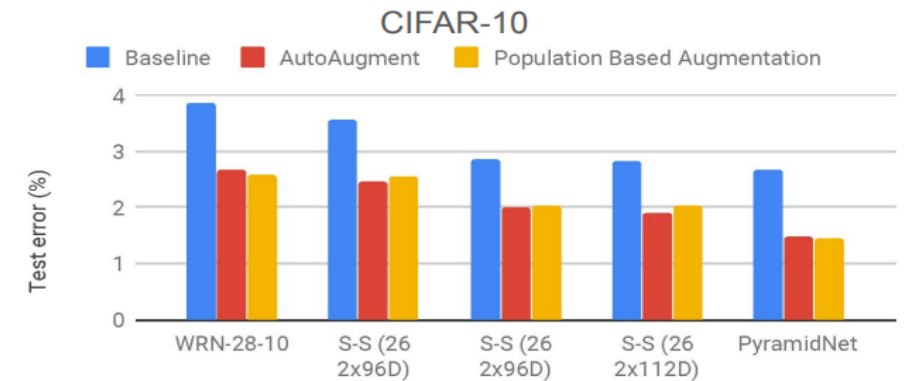
```
Input: Params  $p$ , list of augmentation hyperparameters
for  $param$  in  $p$  do
  if  $\text{random}(0, 1) < 0.2$  then
    Resample  $param$  uniformly from domain
  else
     $amt = [0, 1, 2, 3]$  uniformly at random
    if  $\text{random}(0, 1) < 0.5$  then
       $param = param + amt$ 
    else
       $param = param - amt$ 
    end if
    Clip  $param$  to stay in domain
  end if
end for
```

03 | Experiments

Accuracy and Computational Cost

- AA와 비교하여 일부 우수하거나 유사한 성능을 보여줌
- 연산량을 획기적으로(500배 또는 그 이상)으로 줄임
 - ✓ AA : 15000개 자녀모델 x 120epoch = 1.8m epoch / PBA 16개 자녀모델 x 120epoch = 3200epoch
 - ✓ 자녀모델을 동일 GPU에서 학습할 수 있어 자원의 절약도 가능

Dataset	Model	Baseline	Cutout	AA	AA*	PBA
CIFAR-10	Wide-ResNet-28-10	3.87	3.08	2.68		2.58 ± 0.062
	Shake-Shake (26 2x32d)	3.55	3.02	2.47		2.54 ± 0.10
	Shake-Shake (26 2x96d)	2.86	2.56	1.99		2.03 ± 0.11
	Shake-Shake (26 2x112d)	2.82	2.57	1.89		2.03 ± 0.080
	PyramidNet+ShakeDrop	2.67	2.31	1.48		1.46 ± 0.077
Reduced CIFAR-10	Wide-ResNet-28-10	18.84	17.05	14.13		12.82 ± 0.26
	Shake-Shake (26 2x96d)	17.05	13.40	10.04		10.64 ± 0.22
CIFAR-100	Wide-ResNet-28-10	18.8	18.41	17.09		16.73 ± 0.15
	Shake-Shake (26 2x96d)	17.05	16.00	14.28		15.31 ± 0.28
	PyramidNet+ShakeDrop	13.99	12.19	10.67		10.94 ± 0.094
SVHN	Wide-ResNet-28-10	1.50	1.40	1.07	1.13 ± 0.024	1.18 ± 0.022
	Shake-Shake (26 2x96d)	1.40	1.20	1.02	1.10 ± 0.032	1.13 ± 0.029
Reduced SVHN	Wide-ResNet-28-10	13.21	32.5	8.15		7.83 ± 0.22
	Shake-Shake (26 2x96d)	13.32	24.22	5.92		6.46 ± 0.13



Dataset	Value	Previous Best	AA	PBA
CIFAR-10	GPU Hours	-	5000	5
	Test Error	2.1	1.48	1.46
CIFAR-100	GPU Hours	-	0*	0*
	Test Error	12.2	10.7	10.9
SVHN	GPU Hours	-	1000	1
	Test Error	1.3	1.0	1.1

03 | Experiments

Ablation Study

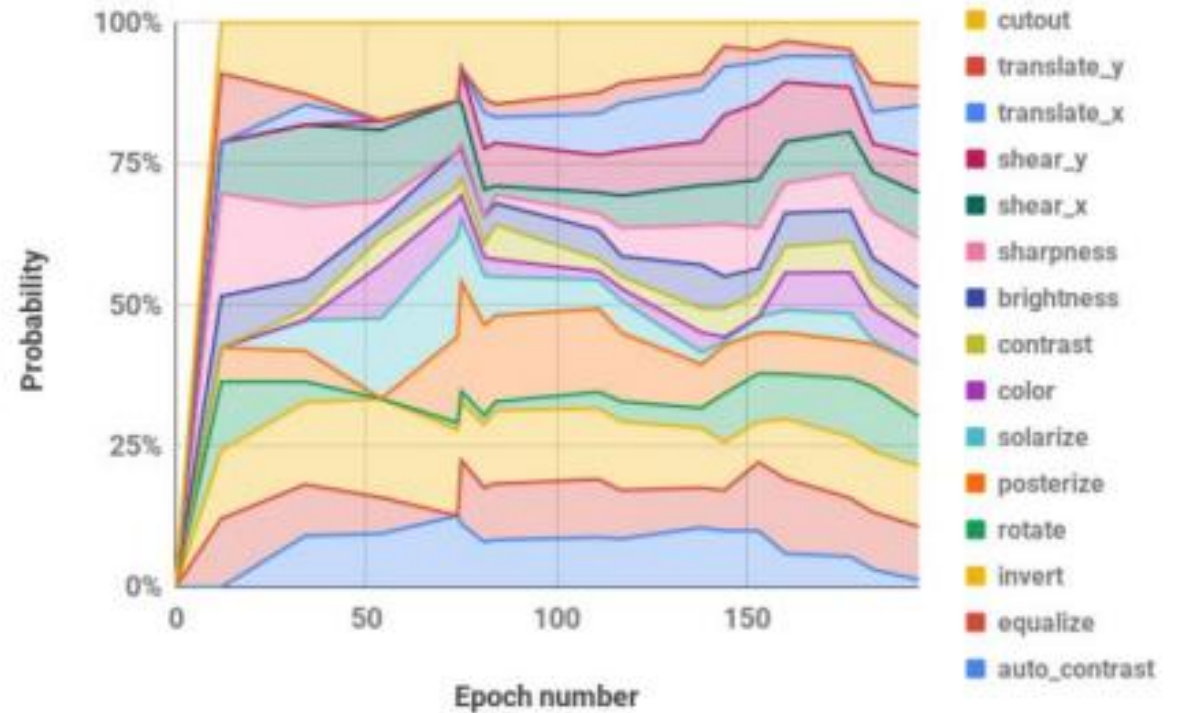
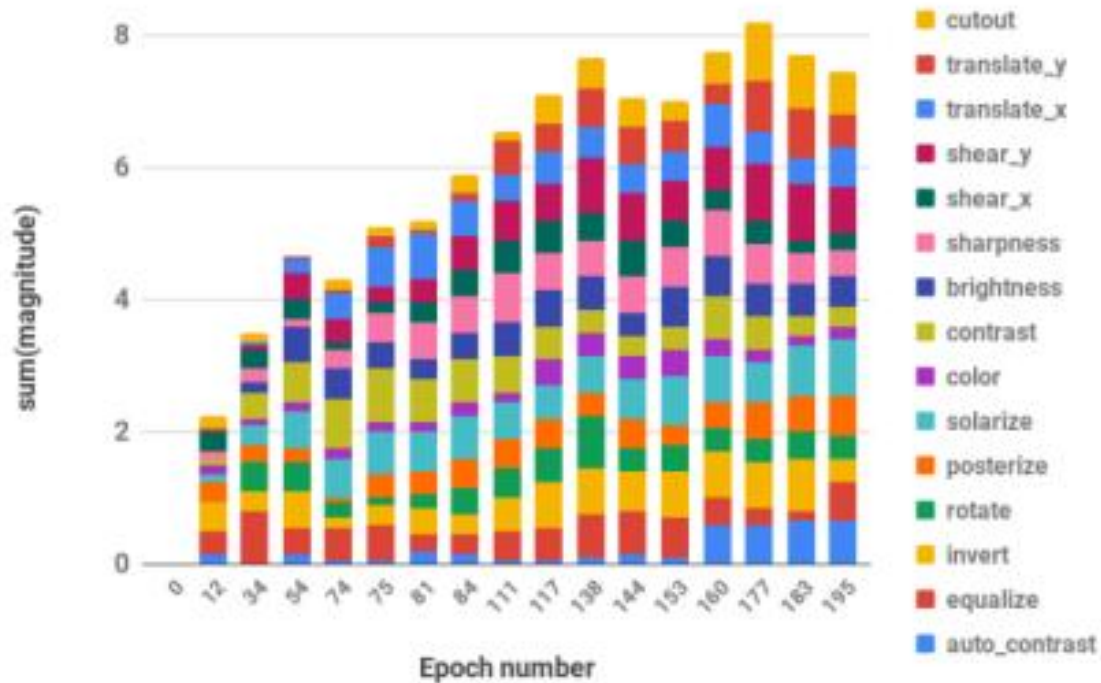
- 마지막 단계 정책을 훈련 내내 고정해서 사용했을때나, 섞어서 사용할 경우 성능이 저하됨을 확인
- 훈련 초기에는 쉬운 증강을 후반부로 갈수록 어려운 증강을 적용하는 것이 효과적이라고 판단됨

Model	Cutout	Fixed Policy	Order-shuffled	Fully-shuffled	PBA
Wide-ResNet-28-10	3.08	2.76	2.66	2.89	2.576 ± 0.062
Shake-Shake (26 2x32d)	3.02	2.73	-	-	2.54 ± 0.10
Shake-Shake (26 2x96d)	2.56	2.33	-	-	2.03 ± 0.11
Shake-Shake (26 2x112d)	2.57	2.09	-	-	2.03 ± 0.080
PyramidNet+ShakeDrop	2.31	1.55	-	-	1.46 ± 0.077

- 증강 연산의 수나 적용 확률을 일부 조정할 경우 성능향상의 여지가 있을거라 보고 있으며(실험 미실시)
성능을 중간 테스트 하는 epoch범위를 2나 4로 조정해도 성능의 변화는 없었음

03 | Experiments

- 130epoch 근처에서 증강의 강도가 일관되어지는 모습을 보였음
- 학습의 종반에 갈수록 적용강도의 그래프가 평탄해지는 모습을 보이고 있음(안정화)



04 | Conclusion

- 본 논문을 한마디로 정의하면 무엇인가?
 - ✓ 학습 성능을 높일 수 있는 증강기법 적용 스케줄을 자동으로 설정할 수 있는 프레임워크 제시
- 본 논문에서 놀랍고, 새로운 점은 무엇인가?
 - ✓ AA와 비교하여 매우 낮은 계산량으로 유사한 성능을 달성
 - ✓ 결정된 스케줄을 분석하여 학습과정별로 효과적인 증강 기법을 찾아낼 수 있음
- 본인이 예상한 바와 어떤 점이 다른가?
 - ✓ 성능측면에서 기존 기법에 비해 우수한 성과를 보이지 못함
 - ✓ 파운데이션 모델의 가중치도 복제하기 때문에 모델의 가중치와 증강기법간의 연관관계를 무시
- 본 논문과 관련된 본인의 아이디어는 무엇인가?
 - ✓ 증강 정책을 랜덤하게 배정하는 것 보다 효과적인 증강정책을 학습하는 모델을 추가하여 성능 향상도모

04 | 동영상 강의

• Overfitting

Data-level solution

- Increase the size of training data to better approximate the true data distribution.

1. Collect more



- ☐ Expensive
- ☐ EXPENSIVE.
- ☐ *E.X.P.E.N.S.I.V.E.*

2. Synthesize



- ☐ Complicated
- ☐ Little variation
- ☐ Mode collapse

3. Augment



- ☐ Simple
- ☐ Well-studied
- ☐ Which is optimal?

7/31



—• Data Augmentation

How much?

Semantically Invariant Transformation


Adverb. 의미상, 의미론적으로 Adjective. 변함없는, 변치 않는

=

Transformations should preserve class labels.

Q&A

감사합니다.